

BTS Services Informatiques aux Organisations (SIO)

Session 2026

Nom du candidat : SEILLER

Prénom du candidat : Julian

Parcours : SLAM

Fiche de réalisation professionnelle N°12

Automatisation des tests unitaires et intégration continue (CI/CD)

Période de réalisation : 1ère et 2ème année de BTS

Type de réalisation : En formation (Lycée Louise Michel)

Mode de réalisation : En autonomie

Sommaire

Contexte Organisationnel.....	3
Problématique de gestion.....	3
Choix de la solution mise en œuvre.....	3
Réalisation de la solution.....	4
a - Prérequis et ressources à disposition.....	4
b - Mise en œuvre de la solution.....	4
c - Tests.....	4
i - Cas d'utilisation 1 : Test unitaire d'un algorithme métier	4
ii - Cas d'utilisation 2 : Test d'intégration d'une route API (Supertest).....	5
iii - Cas d'utilisation 3 : Déclenchement de l'Intégration Continue (GitHub Actions).....	5
Conclusions.....	5
Annexes.....	5
Compétences associées.....	6

Contexte Organisationnel

Cette dernière réalisation s'inscrit dans la continuité du projet "Collège Asimov" (développement d'une API REST en Node.js pour la gestion scolaire). Une fois l'API développée, il était indispensable de s'assurer de sa fiabilité avant de la livrer au client (la DSI de la Fondation Asimov) et de garantir que les futures mises à jour ne casseraient pas les fonctionnalités existantes.

Problématique de gestion

Lors des premières phases de développement, je testais mon code manuellement via des outils comme Postman. Cette méthode posait plusieurs problèmes :

- **Chronophagie** : Tester manuellement chaque route (GET, POST, PUT, DELETE) à chaque modification du code prenait énormément de temps.
- **Risque de régression** : En ajoutant le module de "génération PDF", j'ai involontairement altéré la logique de calcul des moyennes, ce qui n'a pas été détecté immédiatement. Il fallait donc mettre en place un filet de sécurité : un système capable de tester automatiquement l'ensemble du code en quelques secondes, et bloquer le déploiement si une erreur est détectée.

Choix de la solution mise en œuvre

Pour implémenter cette démarche d'Assurance Qualité (QA) et de DevOps, j'ai sélectionné les outils standards de l'écosystème JavaScript :

- **Jest** : Framework de test très populaire pour JavaScript, permettant de créer des suites de tests unitaires claires et d'obtenir un rapport de couverture de code (Code Coverage).
- **Supertest** : Une bibliothèque complémentaire utilisée pour simuler des requêtes HTTP vers mon serveur Express.js sans avoir besoin de le lancer manuellement.
- **GitHub Actions** : Un outil d'Intégration Continue (CI). J'ai configuré un script pour que le serveur GitHub lance automatiquement tous les tests Jest à chaque fois que je pousse (commit) du nouveau code sur le dépôt.

Réalisation de la solution

a - Prérequis et ressources à disposition

- Code source de l'API REST Node.js (Projet Asimov).
- Gestionnaire NPM pour installer les dépendances de développement (jest, supertest).
- Dépôt GitHub distant.

b - Mise en œuvre de la solution

- **Configuration de l'environnement de test** : J'ai isolé la base de données de test de la base de production pour éviter d'insérer des fausses données (Mocking). J'ai ajouté un script de test dans le fichier package.json.
- **Écriture des Tests Unitaires** : J'ai rédigé des tests isolés pour vérifier la logique métier pure, comme la fonction mathématique qui calcule la moyenne d'un élève en fonction des coefficients, en lui passant des valeurs fictives et en vérifiant le retour.
- **Écriture des Tests d'Intégration** : Avec Supertest, j'ai programmé des requêtes HTTP automatisées. Le script simule l'envoi d'un JSON (ex: création d'un élève) et utilise des assertions (expect) pour vérifier que le serveur répond bien avec un code HTTP 201 (Créé).
- **Mise en place du workflow CI/CD** : J'ai créé un fichier de configuration YAML dans le dossier .github/workflows. Ce fichier ordonne aux serveurs de GitHub de créer un environnement virtuel Ubuntu, d'installer Node.js, et de lancer la commande npm test à chaque modification du code.

c - Tests

i - Cas d'utilisation 1 : Test unitaire d'un algorithme métier

- **Entrée / Action** : Le script de test envoie les notes [15, 10, 20] avec des coefficients identiques à la fonction de calcul.
- **Résultat attendu** : La fonction d'assertion (expect) vérifie que le résultat retourné est strictement égal à 15. Le test passe au vert.

ii - Cas d'utilisation 2 : Test d'intégration d'une route API (Supertest)

- **Entrée / Action :** Le script Supertest tente de créer un élève (requête POST) en omettant volontairement le champ obligatoire "Nom".
- **Résultat attendu :** Le test valide que l'API est robuste : il s'attend à recevoir un code d'erreur HTTP 400 (Bad Request) et un message JSON spécifique. Si l'API retourne autre chose, le test échoue.

iii - Cas d'utilisation 3 : Déclenchement de l'Intégration Continue (GitHub Actions)

- **Entrée / Action :** Je pousse une nouvelle mise à jour de mon code (git push) sur la branche principale de mon dépôt GitHub.
- **Résultat attendu :** L'onglet "Actions" de GitHub s'active. Les tests s'exécutent sur le serveur distant en quelques secondes. Un badge vert "Passing" s'affiche sur le dépôt, validant que mon nouveau code n'a causé aucune régression.

Conclusions

La mise en place de tests automatisés a radicalement changé ma façon de programmer. J'ai sécurisé le cycle de développement en anticipant les régressions logicielles (approche Test-Driven Development).

L'intégration de GitHub Actions m'a permis d'automatiser l'assurance qualité (QA) : je garantis désormais de livrer à la production une application fiable et rigoureusement testée à chaque commit. C'est une compétence indispensable pour intégrer une équipe de développement professionnelle.

Annexes

- **Annexe 1 :** Capture d'écran du fichier «package.json» montrant «jest» et «supertest» dans les «devDependencies» ainsi que le script « test » : « jest »

```
{
  "name": "college-asimov",
  "version": "1.0.0",
  "description": "API REST pour la gestion du collège Asimov",
  "main": "src/app.js",
  "scripts": {
    "start": "node src/app.js",
    "test": "jest",
    "dev": "nodemon src/app.js"
  },
  "keywords": [
    "bts",
    "sio",
    "api",
    "jest"
  ],
  "license": "ISC",
  "dependencies": {
    "bcryptjs": "^3.0.3",
    "cors": "^2.8.6",
    "dotenv": "^17.4.2",
    "express": "^4.19.2",
    "express-rate-limit": "^8.5.1",
    "helmet": "^8.1.0",
    "jsonwebtoken": "^9.0.3",
    "pdfkit": "^0.18.0",
    "sequelize": "^6.37.8",
    "sqlite3": "^6.0.1"
  },
  "devDependencies": {
    "jest": "^30.4.2",
    "supertest": "^7.2.2"
  }
}
```

- **Annexe 2 :** Capture d'écran du fichier de configuration GitHub Actions « [github/workflows/node-ci.yml](#) »

```
name: Node.js CI

on:
  push:
    branches: [ "main", "master" ]
  pull_request:
    branches: [ "main", "master" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
      - name: Use Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm install
      - run: npm test
```

Compétences associées

N°	Compétences associées
B1.5.1	Réaliser les tests d'intégration et d'acceptation d'un service (Jest / Supertest)
B3.5.3b	Assurer la maintenance corrective et évolutive (Prévention des régressions)
B1.5.2	Gérer le patrimoine informatique (Déploiement continu via GitHub Actions)